TALLER DE GO · 42 MÁLAGA · SESIÓN 2 · 12 DE NOVIEMBRE DE 2026

Bloque 3 · Concurrencia y transacciones

La taquilla tiene una regla: el stock nunca baja de cero. Tu `Reserve` la cumple cuando las reservas llegan una detrás de otra. En este bloque llegan cien a la vez, y deja de cumplirla.

Primero ves el fallo. Después lo arreglas con un `Mutex`, que sirve mientras haya una sola API. Al final lo arreglas con una transacción, que sirve también cuando hay varias: cinco líneas en `Reserve`.

Lo que viene hecho

El test: cien reservas en el mismo instante

```
start := make(chan struct{})
var wg sync.WaitGroup
var succeeded atomic.Int32
for i := 0; i < clients; i++ {       // clients = 100
    wg.Add(1)
    go func() {
        defer wg.Done()
        <-start
        if _, err := s.Reserve("p1", 1); err == nil {
            succeeded.Add(1)
        }
    }()
}
close(start)
wg.Wait()
```

Está en `oversell_test.go`. Crea un producto con 10 unidades y hace 100 reservas de una unidad, todas a la vez.

- `go func() { ... }()` lanza una goroutine: la función corre a la vez que el resto del programa.
- Las cien goroutines se quedan esperando en `<-start`. `close(start)` las suelta todas en el mismo instante.
- `wg.Wait()` espera a que terminen las cien.
- `succeeded` cuenta las reservas que entraron. Es un contador atómico porque lo tocan cien goroutines a la vez.

Al final el test exige dos cosas: que hayan entrado exactamente 10 reservas y que el stock sea 0.

Por qué falla tu `Reserve`

Tu `Reserve` hace tres cosas, cada una en una sentencia: lee el stock, lo compara y lo resta. Entre la lectura y la resta cabe otra petición:

```
Queda 1 unidad.

    petición A                  petición B
    ------------------------    ------------------------
    lee el stock: 1
                                lee el stock: 1
    1 >= 1, sigue
                                1 >= 1, sigue
    resta 1: el stock es 0
                                resta 1: el stock es -1
```

Cada sentencia es correcta. El fallo es que B decide con un dato que ya no es verdad cuando escribe. Con una petición detrás de otra no pasa nunca: por eso los tests de los bloques 1 y 2 pasan.

El arreglo: una transacción

Una transacción junta varias sentencias para que la base de datos las trate como una sola: o se hacen todas, o no se hace ninguna. En Go tiene esta forma:

```go
tx, err := s.db.Begin()      // empieza la transacción
if err != nil {
    return nil, err
}
defer tx.Rollback()          // si salimos antes del Commit, se deshace todo

// ... las mismas consultas, con tx en vez de s.db:
//     tx.QueryRow(...)
//     tx.Exec(...)

if err := tx.Commit(); err != nil {   // se guarda todo a la vez
    return nil, err
}
```

- `tx` tiene los mismos métodos que `s.db`: `QueryRow`, `Exec` y `Query`. Lo que se hace con `tx` va dentro de la transacción; lo que se hace con `s.db`, fuera.
- `Commit` guarda todo. `Rollback` lo deshace todo. Después de un `Commit`, el `Rollback` del `defer` no hace nada: por eso se puede dejar siempre.
- Mientras una transacción está abierta, la siguiente espera su turno. Lo pide el parámetro `_txlock=immediate`, que está en `OpenStore` desde el bloque 1.

Ver el fallo

Antes de arreglar nada, dos comprobaciones. En ninguna se escribe código.

El test falla

```
go test -run TestConcurrentReserves -v
```

Falla con un mensaje como este. Los números cambian en cada ejecución:

```
oversold: stock ended at -17 and 27 reservations succeeded; only 10 units existed
```

`-race` no lo ve

```
go test -race -run TestConcurrentReserves -v
```

Falla igual y no aparece ningún `WARNING: DATA RACE`. `-race` vigila la memoria del programa: dos goroutines tocando la misma variable. Aquí no comparten ninguna variable; comparten una fila de la base de datos, y eso `-race` no lo mira.

Paso a paso

Cada paso empieza por el test que lo comprueba. El del paso 1 ya lo has visto fallar.

Paso 1 · Un `Mutex` en `Reserve` (lo hacemos juntos)

```
go test -run TestConcurrentReserves -v
```

Un `sync.Mutex` es un cerrojo. Entre `Lock` y `Unlock` solo puede estar una goroutine; las demás esperan en `Lock` a que salga.

- [ ] Añade el campo `mu sync.Mutex` al struct `Store` y `"sync"` a los `import`. No hay que inicializarlo: recién creado ya es un cerrojo abierto.
- [ ] Al principio de `Reserve`, `s.mu.Lock()` y `defer s.mu.Unlock()`.

```
type Store struct {
    db *sql.DB
    mu sync.Mutex
}

func (s *Store) Reserve(productID string, qty int) (*Reservation, error) {
    s.mu.Lock()
    defer s.mu.Unlock()
    // ... lo que ya tenías, tal cual
}
```

Pasa: las cien goroutines entran en `Reserve` de una en una, así que ninguna decide con un stock viejo.

Compruébalo también con la API de verdad. En una terminal, arráncala sobre un fichero nuevo:

```
rm -f demo.db
go run ./cmd/api -db demo.db
```

En otra, lanza cien reservas a la vez con `cmd/oversell`:

```
go run ./cmd/oversell -n 100 http://localhost:8080
```

```
product load-1794495166761 created with stock 10
100 requests: 10 created (201), 90 out of stock (409)
stock now: {"id":"load-1794495166761","name":"Load test","stock":0}
```

`cmd/oversell` crea un producto nuevo con 10 unidades cada vez que lo lanzas. Puedes repetir la carga las veces que quieras sin reiniciar nada.

Dos APIs sobre la misma base de datos

Una API en producción no suele correr sola. Hay varias copias del mismo programa, las réplicas, para repartir la carga y para que la caída de una no lo pare todo. Todas usan la misma base de datos.

Deja la API como está. En una segunda terminal, arranca otra réplica en otro puerto, sobre el mismo fichero:

```
go run ./cmd/api -db demo.db -addr :8081
```

En la tercera, la misma carga repartida entre las dos:

```
go run ./cmd/oversell -n 100 http://localhost:8080 http://localhost:8081
```

```
product load-1794495166761 created with stock 10
100 requests: 11 created (201), 89 out of stock (409)
stock now: {"id":"load-1794495166761","name":"Load test","stock":-1}
```

Se ha vendido una entrada que no existía. No pasa en todas las ejecuciones: si te salen 10 y stock 0, lanza la carga otra vez.

El `Mutex` vive en la memoria de un programa. Cada réplica tiene el suyo y ninguna sabe del de la otra: dentro de cada una las reservas van de una en una, pero una de cada réplica puede leer el stock a la vez. Con más réplicas se cuelan más. Lo único que comparten es la base de datos, así que el arreglo tiene que estar ahí.

Paso 2 · La transacción en `Reserve`

```
go test -run TestConcurrentReserves -v
```

- [ ] Quita el `Mutex`: el campo, el import y las dos líneas de `Reserve`. Lanza el test: vuelve a fallar.

- [ ] Mete en una misma transacción todo lo que `Reserve` hace con la base de datos: la lectura del stock y las dos escrituras. La forma está más arriba, en "El arreglo: una transacción".

Si el test sigue fallando, alguna consulta se ha quedado fuera de la transacción.

Pasa: entran exactamente 10 reservas y el stock acaba en 0.

Después, las dos APIs otra vez. Páralas con Ctrl+C y arráncalas de nuevo, para que usen el código que acabas de escribir, y repite la carga varias veces:

```
go run ./cmd/oversell -n 100 http://localhost:8080 http://localhost:8081
```

```
product load-1794495193310 created with stock 10
100 requests: 10 created (201), 90 out of stock (409)
stock now: {"id":"load-1794495193310","name":"Load test","stock":0}
```

Ahora el turno lo da la base de datos, y vale igual con una API, con dos o con veinte.

Paso 3 · `Cancel` en transacción

```
go test -run TestConcurrentCancels -v
```

El test hace 10 reservas y lanza 4 cancelaciones a la vez sobre cada una. Falla con un mensaje como este:

```
stock returned more than once: it ended at 122 and 32 cancellations succeeded; only 100
units existed
```

Es el mismo fallo que en `Reserve`: varias cancelaciones leen la reserva todavía `Held` y cada una devuelve el stock por su cuenta.

- [ ] Haz lo mismo en `Cancel`: la lectura de la reserva y las dos escrituras, dentro de una misma transacción.

`s.Reservation` lee con `s.db`, es decir, fuera de la transacción. Aquí la reserva hay que leerla con `tx`: `scanReservation` acepta lo que devuelve `tx.QueryRow`.

Comprobar la sesión entera

```
go test ./...
go test -race ./...
```

Has terminado cuando pasan los dos. Lo que tienes ahora:

- Un `Store` que guarda en un fichero SQLite con `database/sql`: los datos sobreviven al reinicio.
- Una API HTTP con `net/http` que habla JSON y responde a cada error con su código.
- Una reserva que no vende de más aunque lleguen cien peticiones a la vez, con una API o con varias.

El material, los checkpoints y la solución están en el repo del taller: `github.com/GoMalaga/workshops`.